

TP Développement Web & Administration Linux

Déploiement d'une application PHP/MySQL sur Apache

Année : 3A-STI

Durée : 4 heures

Environnement : Debian / Ubuntu Server

Responsable : Ahmad ADELL, ahmad.adell@insa-cvl.fr

17 mars 2026

Table des matières

1	Introduction et Objectifs	3
1.1	Présentation du TP	3
1.2	Rappels théoriques	3
1.2.1	PHP (PHP : Hypertext Preprocessor)	3
1.2.2	MySQL	3
1.2.3	Le rôle d'un serveur web (Apache)	3
2	Installation de la stack LAMP (Linux, Apache, MySQL, PHP)	3
2.1	Prérequis système	4
2.2	Installation d'Apache HTTP Server	4
2.3	Installation de MySQL (MariaDB)	4
2.4	Installation de PHP et des extensions nécessaires	5
2.5	Vérification de l'installation	6
2.6	Récapitulatif des services	6
3	Création de l'application web	7
3.1	Structure du projet	7
3.2	Étape 1 : Création de la base de données	7
3.3	Étape 2 : Fichier de connexion (db.php)	8
3.4	Étape 3 : Page d'accueil (index.php)	8
3.5	Étape 4 : Page de connexion (login.php)	10
3.6	Étape 5 : Déconnexion (logout.php)	11
4	Déploiement dans le HOME utilisateur	11
4.1	Pourquoi éviter /var/www/html ?	11
5	Lien symbolique vers /var/www	12
5.1	Concept de lien symbolique	12

6	Configuration Apache : VirtualHost	13
6.1	Qu'est-ce qu'un VirtualHost ?	13
7	Activation du site et redémarrage	14
7.1	Activation avec a2ensite	14
7.2	Reload vs Restart	14
8	Configuration DNS locale (/etc/hosts)	15
8.1	Résolution de noms	15
9	Modules Apache essentiels	15
9.1	mod_rewrite (URL rewriting)	15
9.2	mod_ssl (HTTPS)	15
10	Sécurisation et bonnes pratiques	16
10.1	Permissions des fichiers	16
10.2	Protection des fichiers sensibles	16
10.3	Structure sécurisée recommandée	17
11	Vérifications et tests	17
11.1	Checklist de validation	17
12	Exercices supplémentaires	18
12.1	Exercice 1 : Ajouter une page de profil	18
12.2	Exercice 2 : URL Rewriting avancé	18
12.3	Exercice 3 : HTTPS complet	18
13	Erreurs fréquentes et solutions	19
14	Conclusion	19
14.1	Pour aller plus loin	19
A	Annexe : Commandes récapitulatives	20

1 Introduction et Objectifs

1.1 Présentation du TP

Dans ce TP, vous allez créer une application web complète en **PHP/MySQL**, puis l'installer sur un serveur **Apache HTTP Server** sous Debian. Ce travail pratique combine compétences de développement et d'administration système.

💡 À retenir

Ce TP vous permettra de comprendre le **flux complet** d'une requête web : depuis le navigateur client jusqu'à la base de données, en passant par le serveur web et l'interpréteur PHP.

1.2 Rappels théoriques

1.2.1 PHP (PHP : Hypertext Preprocessor)

PHP est un langage de script **côté serveur** exécuté par le serveur web avant l'envoi de la réponse au navigateur. Le client ne reçoit jamais le code PHP, uniquement le résultat HTML généré.

1.2.2 MySQL

Système de gestion de base de données relationnelle (SGBDR). Il stocke les données de l'application de manière structurée et persistante.

1.2.3 Le rôle d'un serveur web (Apache)

Apache HTTP Server est un logiciel qui :

- Écoute les requêtes HTTP sur le port 80 (HTTP) ou 443 (HTTPS)
- Localise les fichiers demandés sur le disque
- Exécute les scripts PHP si nécessaire
- Retourne la réponse au client

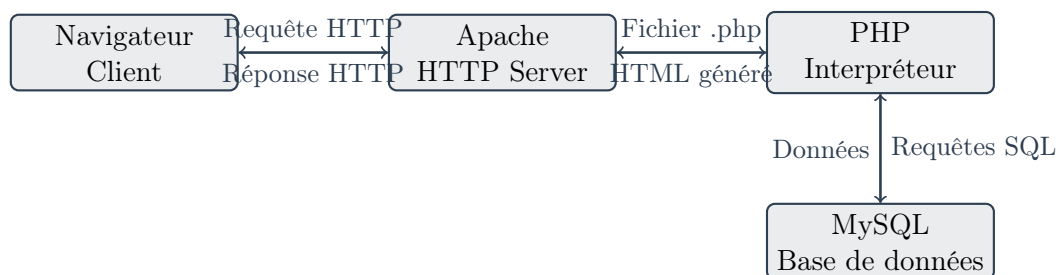


FIGURE 1 – Flux d'une requête web dynamique

2 Installation de la stack LAMP (Linux, Apache, MySQL, PHP)

Avant de commencer le développement, nous devons installer l'ensemble des logiciels nécessaires au fonctionnement de notre application web. Cette ensemble est communément appelé **stack LAMP**.

2.1 Prérequis système

- Un système Debian 11+ ou Ubuntu 20.04+ à jour
- Un accès root ou sudo
- Une connexion Internet active

2.2 Installation d'Apache HTTP Server

Apache est le serveur web le plus utilisé au monde. Il gère les requêtes HTTP et sert les pages web aux clients.

>_ Installation et démarrage d'Apache

```
1 # Mise a jour des depots
2 sudo apt update
3
4 # Installation d'Apache
5 sudo apt install apache2 -y
6
7 # Demarrage du service et activation au boot
8 sudo systemctl start apache2
9 sudo systemctl enable apache2
10
11 # Verification du statut
12 sudo systemctl status apache2
```

💡 À retenir

Commandes essentielles Apache :

- `sudo systemctl start apache2` : Démarrer le serveur
- `sudo systemctl stop apache2` : Arrêter le serveur
- `sudo systemctl restart apache2` : Redémarrer le serveur
- `sudo systemctl reload apache2` : Recharger la configuration
- `sudo systemctl status apache2` : Voir le statut actuel

Après installation, testez en ouvrant un navigateur et en accédant à `http://localhost` ou `http://127.0.0.1`. Vous devriez voir la page par défaut d'Apache.

2.3 Installation de MySQL (MariaDB)

MariaDB est un fork libre de MySQL, fully compatible et souvent préféré sur les distributions Debian récentes.

>_ Installation et sécurisation de MariaDB

```
1 # Installation du serveur et client MariaDB
2 sudo apt install mariadb-server mariadb-client -y
3
4 # Demarrage du service
5 sudo systemctl start mariadb
6 sudo systemctl enable mariadb
7
8 # Script de securisation (SUPER IMPORTANT !)
9 sudo mysql_secure_installation
```

Le script `mysql_secure_installation` vous demandera de :

1. Définir un mot de passe root pour MySQL
2. Supprimer l'utilisateur anonyme
3. Désactiver la connexion root à distance
4. Supprimer la base de test
5. Recharger les privilèges

⚠ Attention Sécurité critique

Ne pas confondre le mot de passe root MySQL avec le mot de passe root système ! Ce sont deux comptes distincts. Le root MySQL a tous les droits sur les bases de données uniquement.

2.4 Installation de PHP et des extensions nécessaires

PHP est le langage de script côté serveur. Nous installons également les extensions permettant la connexion à MySQL et diverses fonctionnalités utiles.

>_ Installation de PHP et extensions

```
1 # Installation de PHP et des extensions courantes
2 sudo apt install php libapache2-mod-php php-mysql php-cli \
3     php-curl php-gd php-mbstring php-xml php-zip php-json -y
4
5 # Verification de la version installée
6 php -v
7
8 # Redemarrage d'Apache pour prendre en compte PHP
9 sudo systemctl restart apache2
```

Description des extensions installées :

Package	Description
php	Interpréteur PHP de base
libapache2-mod-php	Module Apache pour exécuter PHP (mod_php)
php-mysql	Extension PDO et MySQLi pour connexion aux bases de données
php-cli	Interface en ligne de commande PHP
php-curl	Bibliothèque pour requêtes HTTP externes
php-gd	Manipulation d'images (GD Library)
php-mbstring	Gestion des chaînes multi-octets (UTF-8)
php-xml	Support XML et DOM
php-zip	Compression/décompression ZIP
php-json	Support JSON (souvent intégré par défaut)

2.5 Vérification de l'installation

Créez un fichier de test pour vérifier que PHP fonctionne correctement avec Apache :

>_ Test de fonctionnement PHP

```
1 # Créer le fichier de test
2 echo "<?php phpinfo(); ?>" | sudo tee /var/www/html/info.php
3
4 # Modifier les permissions
5 sudo chmod 644 /var/www/html/info.php
```

Accédez ensuite à <http://localhost/info.php> dans votre navigateur. Vous devriez voir une page détaillant la configuration PHP.

⚠ Attention Suppression du fichier info.php

Important : Le fichier `info.php` expose des informations sensibles sur votre serveur. Supprimez-le immédiatement après vérification :

```
1 sudo rm /var/www/html/info.php
```

2.6 Récapitulatif des services

Vérifiez que tous les services sont actifs :

```
1 # Vérifier Apache
2 sudo systemctl is-active apache2
3
4 # Vérifier MariaDB/MySQL
5 sudo systemctl is-active mariadb
6
7 # Vérifier les versions
8 apache2 -v
9 mysql --version
10 php -v
```

? Question de réflexion

Question : Pourquoi faut-il redémarrer Apache après l'installation de PHP, alors que ce n'est pas nécessaire après l'installation d'Apache lui-même ?

3 Création de l'application web

Nous allons développer une **mini-application de gestion de messages** (guestbook) avec authentification simple.

3.1 Structure du projet

>_ Structure des fichiers

```
1 /home/dev/www/
2     index.php      # Page home et liste des messages
3     login.php      # Page de connexion
4     logout.php     # Deconnexion
5     db.php         # Configuration base de données
6     config.php     # Configuration db (non accessible)
7     css/
8         style.css  # Feuille de styles
```

3.2 Étape 1 : Création de la base de données

Connectez-vous à MySQL en root :

```
1 sudo mysql -u root
```

Créez la base de données et un utilisateur dédié :

```
1 -- Créer la base de données
2 CREATE DATABASE guestbook CHARACTER SET utf8mb4 COLLATE
   utf8mb4_unicode_ci;
3
4 -- Créer un utilisateur dédié (SECURITE IMPORTANTE)
5 CREATE USER 'gb_user'@'localhost' IDENTIFIED BY 'MotDePasseFort123!';
6 GRANT ALL PRIVILEGES ON guestbook.* TO 'gb_user'@'localhost';
7 FLUSH PRIVILEGES;
8
9 -- Selectionner la base
10 USE guestbook;
11
12 -- Table des utilisateurs
13 CREATE TABLE users (
14     id INT AUTO_INCREMENT PRIMARY KEY,
15     username VARCHAR(50) UNIQUE NOT NULL,
16     password VARCHAR(255) NOT NULL,
17     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
18 );
19
20 -- Table des messages
21 CREATE TABLE messages (
```

```

22     id INT AUTO_INCREMENT PRIMARY KEY,
23     user_id INT,
24     content TEXT NOT NULL,
25     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
26     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
27 );
28
29 -- Insérer un utilisateur de test (mot de passe: test123)
30 INSERT INTO users (username, password)
31 VALUES ('admin', '$2y$12$Jfd0lQPbnFpDPLTY/
    rsqmuTkVX3IGDmVcctlAYcxOGXrX4YmC32Xu');
32
33 EXIT;

```

💡 À retenir

Bonnes pratiques sécurité :

- Toujours créer un utilisateur dédié par application (ne jamais utiliser root)
- Les mots de passe doivent être hashés avec `password_hash()` en PHP
- L'utilisateur `gb_user` n'a accès qu'à la base `guestbook`

3.3 Étape 2 : Fichier de connexion (db.php)

> Créer le fichier /home/dev/www/db.php

```

1 <?php
2 // Configuration de la base de données
3 define('DB_HOST', 'localhost');
4 define('DB_NAME', 'guestbook');
5 define('DB_USER', 'gb_user');
6 define('DB_PASS', 'MotDePasseFort123!');
7
8 // Connexion PDO avec gestion d'erreurs
9 try {
10     $pdo = new PDO(
11         "mysql:host=" . DB_HOST . ";dbname=" . DB_NAME . ";charset=utf8mb4",
12         DB_USER,
13         DB_PASS,
14         [
15             PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
16             PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
17             PDO::ATTR_EMULATE_PREPARES => false,
18         ]
19     );
20 } catch (PDOException $e) {
21     die("Erreur de connexion : " . $e->getMessage());
22 }
23 ?>

```

3.4 Étape 3 : Page d'accueil (index.php)


```
1 <?php
2 session_start();
3 require_once 'db.php';
4
5 // Traitement du formulaire
6 if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_SESSION['user_id
7     '])) {
8     $content = trim($_POST['content'] ?? '');
9     if (!empty($content)) {
10         $stmt = $pdo->prepare("INSERT INTO messages (user_id, content
11             ) VALUES (?, ?)");
12         $stmt->execute([$_SESSION['user_id'], $content]);
13         header('Location: index.php');
14         exit;
15     }
16 }
17
18 // Recuperation des messages
19 $stmt = $pdo->query("
20     SELECT m.*, u.username
21     FROM messages m
22     LEFT JOIN users u ON m.user_id = u.id
23     ORDER BY m.created_at DESC
24 ");
25 $messages = $stmt->fetchAll();
26
27 <?>
28 <!DOCTYPE html>
29 <html lang="fr">
30 <head>
31     <meta charset="UTF-8">
32     <title>Livre d'or</title>
33     <style>
34         body { font-family: Arial, sans-serif; max-width: 800px;
35             margin: 0 auto; padding: 20px; }
36         .message { border: 1px solid #ddd; margin: 10px 0; padding:
37             15px; border-radius: 5px; }
38         .meta { color: #666; font-size: 0.9em; }
39         form { margin: 20px 0; }
40         textarea { width: 100%; height: 100px; }
41     </style>
42 </head>
43 <body>
44     <h1>Livre d'or</h1>
45
46     <?php if (isset($_SESSION['username'])): ?>
47         <p>Bienvenue, <strong><?php echo htmlspecialchars($_SESSION['
48             username']); ?></strong>!
49         <a href="logout.php">Deconnexion</a></p>
50
51         <form method="POST">
52             <textarea name="content" placeholder="Votre message..."
53                 required></textarea><br>
54             <button type="submit">Publier</button>
55         </form>
56     <?php else: ?>
57         <p><a href="login.php">Connectez-vous</a> pour laisser un
58             message.</p>
```

```

51     <?php endif; ?>
52
53     <h2>Messages</h2>
54     <?php foreach ($messages as $msg): ?>
55         <div class="message">
56             <p><?php echo nl2br(htmlspecialchars($msg['content']));
57             ?></p>
58             <p class="meta">
59                 Par <?php echo htmlspecialchars($msg['username']) ?? '
60                 Anonyme'; ?>
61                 le <?php echo $msg['created_at']; ?>
62             </p>
63         </div>
64     <?php endforeach; ?>
65 </body>
66 </html>

```

3.5 Étape 4 : Page de connexion (login.php)

```

1  <?php
2  session_start();
3  require_once 'db.php';
4
5  $error = '';
6
7  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
8      $username = trim($_POST['username'] ?? '');
9      $password = $_POST['password'] ?? '';
10
11      if (!empty($username) && !empty($password)) {
12          $stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?
13          ");
14          $stmt->execute([$username]);
15          $user = $stmt->fetch();
16
17          if ($user && password_verify($password, $user['password'])) {
18              $_SESSION['user_id'] = $user['id'];
19              $_SESSION['username'] = $user['username'];
20              header('Location: index.php');
21              exit;
22          } else {
23              $error = "Identifiants incorrects";
24          }
25      }
26  }
27  <?>
28  <!DOCTYPE html>
29  <html lang="fr">
30  <head>
31      <meta charset="UTF-8">
32      <title>Connexion</title>
33  </head>
34  <body>
35      <h1>Connexion</h1>
36      <?php if ($error): ?>
37          <p style="color: red;"><?php echo $error; ?></p>

```

```
37     <?php endif; ?>
38     <form method="POST">
39         <p>
40             <label>Utilisateur :</label>
41             <input type="text" name="username" required>
42         </p>
43         <p>
44             <label>Mot de passe :</label>
45             <input type="password" name="password" required>
46         </p>
47         <button type="submit">Se connecter</button>
48     </form>
49     <p><em>Test : admin / test123</em></p>
50 </body>
51 </html>
```

3.6 Étape 5 : Déconnexion (logout.php)

```
1 <?php
2 session_start();
3 session_destroy();
4 header('Location: index.php');
5 exit;
6 ?>
```

❓ Question de réflexion

Question : Pourquoi utilise-t-on `htmlspecialchars()` avant d'afficher du contenu provenant de la base de données ? Que se passerait-il si on l'oubliait ?

4 Déploiement dans le HOME utilisateur

4.1 Pourquoi éviter `/var/www/html` ?

Par défaut, Apache sert les fichiers depuis `/var/www/html`. Cependant, pour le développement :

- **Séparation des privilèges :** L'utilisateur dev doit pouvoir modifier ses fichiers sans être root
- **Permissions :** `/var/www` appartient à root (www-data), modification complexe
- **Multi-utilisateurs :** Chaque développeur peut avoir son propre espace dans `/home`
- **Backup :** Le home est souvent sauvegardé automatiquement

>_ Création de l'arborescence

```
1 # Creer l'utilisateur dev s'il n'existe pas
2 sudo adduser dev
3
4 # Creer le repertoire www dans le home
5 sudo mkdir -p /home/dev/www
6 sudo chown -R dev:dev /home/dev/www
7 sudo chmod 755 /home/dev/www
8
9 # Placer les fichiers PHP crees precedemment
10 # (Utilisez scp, sftp, ou copie directe)
11 sudo cp -r /chemin/vers/vos/fichiers/* /home/dev/www/
12 sudo chown -R dev:dev /home/dev/www/*
```

💡 À retenir

Permissions importantes :

- 755 pour les répertoires (rwxr-xr-x)
- 644 pour les fichiers PHP (rw-r-r-)
- Apache lit avec l'utilisateur **www-data**, donc "autres" doit avoir lecture

5 Lien symbolique vers /var/www

5.1 Concept de lien symbolique

Un **lien symbolique** (symlink) est un fichier spécial qui pointe vers un autre fichier ou répertoire. C'est un raccourci au niveau du système de fichiers.

>_ Création du lien symbolique

```
1 # Creer le lien symbolique
2 sudo ln -s /home/dev/www /var/www/monsite
3
4 # Verifier
5 ls -la /var/www/
6 # Doit afficher : monsite -> /home/dev/www
```

💡 À retenir

Avantages du lien symbolique :

- **Séparation** : Code source dans /home, point d'accès dans /var/www
- **Flexibilité** : On peut changer la cible sans modifier Apache
- **Multi-versions** : monsite-v1, monsite-v2, bascule instantanée
- **Déploiement** : Mise à jour par simple changement de lien

? Question de réflexion

Question : Quelle est la différence entre un lien symbolique (`ln -s`) et un lien dur (`ln` sans option) ? Pourquoi utilise-t-on un lien symbolique ici plutôt qu'un lien dur ?

6 Configuration Apache : VirtualHost

6.1 Qu'est-ce qu'un VirtualHost ?

Un **VirtualHost** permet d'héberger plusieurs sites web sur une seule machine avec une seule instance Apache. Apache choisit le site en fonction de :

- Le nom de domaine (`ServerName`)
- L'adresse IP et le port

>_ Création du VirtualHost

Créez le fichier `/etc/apache2/sites-available/monsite.conf` :

```
1 sudo nano /etc/apache2/sites-available/monsite.conf
```

Contenu du fichier :

```
1 <VirtualHost *:80>
2     # Nom de domaine du site
3     ServerName monsite.local
4
5     # Email de l'administrateur (apparaît dans les erreurs)
6     ServerAdmin webmaster@monsite.local
7
8     # Racine du site (ou se trouvent les fichiers)
9     DocumentRoot /var/www/monsite
10
11     # Configuration du repertoire
12     <Directory /var/www/monsite>
13         # Autorise le suivi des liens symboliques (important pour
14         nous !)
15         Options FollowSymLinks
16
17         # Autorise les .htaccess (pour mod_rewrite plus tard)
18         AllowOverride All
19
20         # Autorise l'accès à tous
21         Require all granted
22
23         # Définit index.php comme page par défaut
24         DirectoryIndex index.php index.html
25     </Directory>
26
27     # Fichiers de logs spécifiques à ce site
28     ErrorLog ${APACHE_LOG_DIR}/monsite-error.log
29     CustomLog ${APACHE_LOG_DIR}/monsite-access.log combined
30
31     # Configuration PHP (optionnel)
32     <FilesMatch \.php$>
```

```

32     SetHandler application/x-httpd-php
33     </FilesMatch>
34 </VirtualHost>

```

⚠ Attention Piège fréquent

Si vous oubliez `FollowSymLinks`, Apache refusera de suivre le lien symbolique et vous obtiendrez une erreur **"403 Forbidden"** même avec les bonnes permissions !

❓ Question de réflexion

Question : Que se passe-t-il si deux `VirtualHost` ont le même `ServerName` ? Comment Apache choisit-il quel site servir si plusieurs correspondent ?

7 Activation du site et redémarrage

7.1 Activation avec a2ensite

La commande `a2ensite` (Apache 2 Enable Site) crée un lien symbolique de `sites-available` vers `sites-enabled`.

>_ Activation et redémarrage

```

1  # Activer le site
2  sudo a2ensite monsite.conf
3
4  # Désactiver le site par défaut (optionnel mais recommande)
5  sudo a2disssite 000-default.conf
6
7  # Recharger la configuration Apache
8  sudo systemctl reload apache2
9
10 # Ou redémarrer complètement (si reload ne suffit pas)
11 sudo systemctl restart apache2

```

7.2 Reload vs Restart

Commande	Action	Impact
<code>reload</code>	Relit les fichiers de configuration	Sans coupure des connexions existantes
<code>restart</code>	Arrête et redémarre le processus	Coupure temporaire du service

💡 À retenir

Quand utiliser `reload` ou `restart` ?

- `reload` : Modification de `VirtualHost`, activation/désactivation de site
- `restart` : Installation d'un module, changement de ports d'écoute, problème majeur

8 Configuration DNS locale (/etc/hosts)

8.1 Résolution de noms

Pour tester `monsite.local`, votre ordinateur doit savoir que ce nom correspond à l'IP de votre serveur. Comme nous n'avons pas de vrai DNS, nous utilisons le fichier `/etc/hosts`.

⚠ Attention

Le fichier `/etc/hosts` est consulté **avant** les serveurs DNS. C'est une résolution **locale** valable uniquement sur cette machine.

>_ Modification du fichier hosts

```
1 # Editer le fichier hosts
2 sudo nano /etc/hosts
3
4 # Ajouter cette ligne a la fin :
5 127.0.0.1    monsite.local
6
7 # Pour un reseau local, utilisez l'IP du serveur :
8 # 192.168.1.100    monsite.local
```

❓ Question de réflexion

Question : Quelle est la différence entre la résolution via `/etc/hosts` et un vrai serveur DNS ? Quels sont les avantages et inconvénients de chaque méthode ?

9 Modules Apache essentiels

9.1 `mod_rewrite` (URL rewriting)

Permet de réécrire les URLs dynamiquement (ex : `/article/5` au lieu de `/article.php?id=5`).

>_ Activation de `mod_rewrite`

```
1 # Activer le module
2 sudo a2enmod rewrite
3
4 # Recharger Apache
5 sudo systemctl reload apache2
```

Testez avec un fichier `.htaccess` dans `/home/dev/www/` :

```
1 RewriteEngine On
2 RewriteRule ^bonjour$ index.php [L]
```

Accédez à `http://monsite.local/bonjour` → doit afficher l'accueil.

9.2 `mod_ssl` (HTTPS)

Permet le chiffrement SSL/TLS des communications.

>_ Activation SSL

```
1 # Activer le module SSL
2 sudo a2enmod ssl
3
4 # Activer le site SSL par défaut (pour test)
5 sudo a2ensite default-ssl
6
7 # Generer un certificat auto-signe (pour test uniquement)
8 sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
9     -keyout /etc/ssl/private/monsite.key \
10    -out /etc/ssl/certs/monsite.crt \
11    -subj "/CN=monsite.local"
12
13 # Recharger Apache
14 sudo systemctl reload apache2
```

💡 À retenir

Principe du HTTPS :

1. Le serveur présente un certificat (identité)
2. Le navigateur vérifie la validité (signature, date, domaine)
3. Échange de clés pour chiffrer la communication
4. Toutes les données sont chiffrées (confidentialité + intégrité)

10 Sécurisation et bonnes pratiques

10.1 Permissions des fichiers

>_ Vérification des permissions

```
1 # Verifier les permissions actuelles
2 ls -la /home/dev/www/
3
4 # Corriger si necessaire
5 sudo find /home/dev/www -type d -exec chmod 755 {} \;
6 sudo find /home/dev/www -type f -exec chmod 644 {} \;
7
8 # Proteger le fichier de configuration (contient le mot de passe
9    !)
10 chmod 640 /home/dev/www/db.php
11 sudo chown dev:www-data /home/dev/www/db.php
```

10.2 Protection des fichiers sensibles

Ajoutez dans votre .htaccess ou VirtualHost :

```
1 # Interdire l'accès aux fichiers sensibles
2 <FilesMatch "^\.">
3     Order allow,deny
```



```
4     Deny from all
5 </FilesMatch>
6
7 <FilesMatch "(^#.*#|\.(bak|config|dist|fla|inc|ini|log|psd|sh|sql|sw[
   op])|~)$">
8     Order allow,deny
9     Deny from all
10 </FilesMatch>
```

⚠ Attention Erreur critique

Ne jamais laisser `db.php` accessible en lecture publique ! Il contient le mot de passe de la base de données. Vérifiez que `http://monsite.local/db.php` ne retourne pas le contenu du fichier (normalement, PHP l'exécute, mais en cas de panne PHP, le code source serait visible).

10.3 Structure sécurisée recommandée

```
1 /home/dev/
2     www/                                # DocumentRoot (accessible web)
3         index.php
4         login.php
5         css/
6     includes/                            # Hors DocumentRoot (inaccessible
directement)
7         db.php
8         config.php
9     logs/                                # Logs applicatifs
```

Avec cette structure, `db.php` est totalement inaccessible depuis le web.

11 Vérifications et tests

11.1 Checklist de validation

1. Accéder à `http://monsite.local` → Page d'accueil affichée
2. Se connecter avec `admin/test123` → Session active
3. Poster un message → Message enregistré et affiché
4. Vérifier les logs : `sudo tail -f /var/log/apache2/monsite-error.log`
5. Tester `http://monsite.local/db.php` → Doit être vide ou exécuté, jamais le source

>_ Commandes de diagnostic

```
1 # Tester la configuration Apache (sans appliquer)
2 sudo apache2ctl configtest
3
4 # Voir les VirtualHost actifs
5 sudo apache2ctl -S
6
7 # Verifier les ports en ecoute
8 sudo ss -tlnp | grep apache
9
10 # Voir les dernieres erreurs
11 sudo tail -20 /var/log/apache2/error.log
```

12 Exercices supplémentaires

12.1 Exercice 1 : Ajouter une page de profil

Créez `profil.php` qui permet à l'utilisateur connecté de :

- Voir ses informations
- Changer son mot de passe (avec vérification de l'ancien)

12.2 Exercice 2 : URL Rewriting avancé

Transformez `profil.php?id=5` en `/profil/5` avec `mod_rewrite`.

```
1 # Dans .htaccess
2 RewriteEngine On
3 RewriteBase /
4 RewriteRule ^profil/([0-9]+)$ profil.php?id=$1 [L,QSA]
```

12.3 Exercice 3 : HTTPS complet

Configurez votre VirtualHost pour forcer le HTTPS :

```
1 <VirtualHost *:80>
2     ServerName monsite.local
3     Redirect permanent / https://monsite.local/
4 </VirtualHost>
5
6 <VirtualHost *:443>
7     ServerName monsite.local
8     DocumentRoot /var/www/monsite
9     SSLEngine on
10    SSLCertificateFile /etc/ssl/certs/monsite.crt
11    SSLCertificateKeyFile /etc/ssl/private/monsite.key
12    # ... reste de la configuration
13 </VirtualHost>
```

13 Erreurs fréquentes et solutions

Symptôme	Cause probable	Solution
403 Forbidden	Permissions incorrectes ou pas de FollowSymLinks	<code>chmod 755</code> + vérifier Virtual-Host
404 Not Found	Fichier inexistant ou mauvais DocumentRoot	Vérifier le chemin avec <code>ls -la</code>
500 Internal Server Error	Erreur syntaxe dans <code>.htaccess</code> ou PHP	Vérifier <code>/var/log/apache2/error.log</code>
PHP affiché en texte	Module PHP non activé	<code>sudo a2enmod php8.1</code> (adapter version)
Connexion MySQL refusée	Mauvais credentials ou MySQL arrêté	Vérifier <code>db.php</code> + <code>sudo systemctl status mysql</code>
Site par défaut affiché	VirtualHost pas activé ou mauvais ServerName	<code>sudo a2ensite monsite.conf</code> + vérifier hosts

14 Conclusion

Vous avez maintenant déployé une application web complète sur Apache !

💡 À retenir

Compétences acquises :

- Installation et configuration de la stack LAMP
- Développement PHP sécurisé avec PDO et sessions
- Architecture de base de données relationnelle
- Configuration avancée d'Apache (VirtualHost, modules)
- Gestion des permissions Linux
- Concepts de réseau (DNS, HTTP, HTTPS)

14.1 Pour aller plus loin

- **Docker** : Conteneuriser cette application
- **Nginx** : Tester avec un autre serveur web
- **Let's Encrypt** : Obtenir un vrai certificat SSL gratuit
- **PHP-FPM** : Optimiser les performances PHP

A Annexe : Commandes récapitulatives

```
1 # INSTALLATION COMPLETE DE LA STACK LAMP
2 sudo apt update
3 sudo apt install apache2 mariadb-server mariadb-client \
4     php libapache2-mod-php php-mysql php-cli \
5     php-curl php-gd php-mbstring php-xml php-zip php-json -y
6
7 # Sécurisation MySQL
8 sudo mysql_secure_installation
9
10 # Demarrage des services
11 sudo systemctl enable --now apache2 mariadb
12
13 # DEPLOIEMENT
14 sudo mkdir -p /home/dev/www
15 sudo chown dev:dev /home/dev/www
16 sudo ln -s /home/dev/www /var/www/monsite
17
18 # CONFIGURATION APACHE
19 sudo nano /etc/apache2/sites-available/monsite.conf
20 sudo a2ensite monsite.conf
21 sudo a2dissite 000-default.conf
22 sudo a2enmod rewrite ssl
23 sudo systemctl reload apache2
24
25 # SECURITE
26 sudo find /home/dev/www -type d -exec chmod 755 {} \;
27 sudo find /home/dev/www -type f -exec chmod 644 {} \;
28 sudo chmod 640 /home/dev/www/db.php
29
30 # DIAGNOSTIC
31 sudo apache2ctl configtest
32 sudo apache2ctl -S
33 sudo tail -f /var/log/apache2/monsite-error.log
```